

- [64] Derong Xu, Xinhang Li, Ziheng Zhang, Zhenxi Lin, Zhihong Zhu, Zhi Zheng, Xian Wu, Xiangyu Zhao, Tong Xu, and Enhong Chen. Harnessing large language models for knowledge graph question answering via adaptive multi-aspect retrieval-augmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 25570–25578, 2025.
- [65] Angel Yanguas-Gil, Matthew T Dearing, Jeffrey W Elam, Jessica C Jones, Sungjoon Kim, Adnan Mohammad, Chi Thang Nguyen, and Bratin Sengupta. Benchmarking large language models for materials synthesis: The case of atomic layer deposition. *Journal of Vacuum Science & Technology A*, 43(3), 2025.
- [66] Jihong Yim, Oili ME Ylivaara, Markku Ylilammi, Virpi Korpelainen, Eero Haimi, Emma Verkama, Mikko Utriainen, and Riikka L Puurunen. Saturation profile based conformality analysis for atomic layer deposition: aluminum oxide in lateral high-aspect-ratio channels. *Physical Chemistry Chemical Physics*, 22(40):23107–23120, 2020.
- [67] Jihong Yim, Oili M. E. Ylivaara, Markku Ylilammi, Virpi Korpelainen, Eero Haimi, Emma Verkama, Mikko Utriainen, and Riikka L. Puurunen. Comparison of precursor sticking coefficients in ALD Al_2O_3 on high-aspect-ratio structures, 2025. URL <https://orkg.org/comparisons/R1469158>.
- [68] Zhimin Yu, Kang Yuan, Yang Yang, and Jiaming Sun. Polycrystalline er-doped y 3 ga 5 o 12 nanofilms fabricated by atomic layer deposition on silicon at a low temperature and the exploration on electroluminescence performance. *Nanoscale*, 14(29):10540–10548, 2022.
- [69] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models, 2025. URL <https://arxiv.org/abs/2303.18223>.

Appendix

A Experimental Setting 2 – Detailed Technical Description

In this setting, we evaluate the neural upper bound of performance by treating the PDF of each review article as the only source of evidence. Both proprietary and open-weights language models receive the detailed natural-language equivalent of the SPARQL query together with the article PDF (or its extracted text and tables), but *no* direct access to the ORKG representations. The models must therefore internally parse the PDF, locate the relevant passages and tables, and aggregate the required quantities in a single neural inference step. By comparing their outputs against the SPARQL gold standard from Setting 1, we can quantify how closely purely neural, PDF-based querying can approach symbolic performance for precise, table-style scientific questions.

- A.1 *Proprietary language model (neural) querying.* Here we use the proprietary chat interfaces of three state-of-the-art conversational generative AI models—ChatGPT-5.1 (OpenAI, release 12.11.2025)[47], Gemini-3-Pro (Google, release 18.11.2025)[50], and Claude-Sonnet-4.5 (Anthropic, release 29.09.2025)[5]. As of the date of this submission (13.12.2025), these systems represent the most capable commercially hosted LLMs. Each model is provided with the detailed natural-language equivalent of the SPARQL query and the PDF of the review article. This setting tests whether proprietary models, operating as black-box systems with undisclosed PDF-processing workflows, can directly answer precise scientific questions from the PDFs alone. However, proprietary models provide limited experimental control: their internal PDF-parsing pipelines are opaque, their context integration strategies cannot be inspected or modified, and their outputs cannot be reproduced or systematically debugged. To obtain full transparency, reproducibility, and fine-grained experimental control, we therefore complement this setting with open-weights models.
- A.2 *Open-weights language model (neural) querying.* This setting mirrors Setting 2, but instead of proprietary models, we evaluate high-performing open-weights LLMs. We extract text and tabular content from each PDF using pdfplumber, and provide that content as

input to the models. Five open-weights models are evaluated which are: gemma-3-27b-it, llama-3.3-70b-instruct, qwen2.5-vl-72b-instruct, qwen3-30b-a3b-instruct-2507, and mistral-large-instruct. These models were selected due to their demonstrated strength in large-scale tabular reasoning benchmarks [11].

- a) *Context-injected prompting (single-shot document QA)*. In this variant, the full text and all tables of each paper are extracted using `pdf_extractor.PDFExtractor` and concatenated into one long context window. This complete article representation is passed directly to an open-weights model together with the detailed natural-language query, as implemented in `context_injected_doc_qa.py`. The model must answer solely from this unfiltered, full-document context. While this approach does not control noise or redundancy, it guarantees that tables remain intact and all evidence is available to the model.
- b) *Retrieval-augmented generation (RAG) over PDF segments*. As in Setting 3.1, we first extract all text and tables using `PDFExtractor`. Instead of supplying the entire document to the model, we split the extracted text into contiguous segments of approximately 8 000 characters. In our corpus, a segment typically corresponds to one to three PDF pages—long enough to contain most tables, though some wide or multi-page tables may still be split. Each segment is converted into a dense numerical vector (an “embedding”) using a local Ollama encoder (default: `nomic-embed-text:latest`), producing a per-paper vector index, as implemented in `rag_pdf_segments_doc_qa.py`. At query time, the detailed natural-language query is embedded with the same encoder, and cosine similarity [58] is computed against all segment embeddings. We retrieve the single top-ranked segment ($\text{TOP_K} = 1$) and expand the selection to include neighboring segments in index order, which helps reconstruct table content that may straddle segment boundaries. Only these retrieved segments, together with the detailed query, are provided to the model.

Potential limitations. Because RAG introduces an explicit retrieval stage, its behavior depends not only on the language model but also on how we index each paper and select segments for a given query. In our implementation, these design choices introduce several potential failure modes:

- *Segmentation effects.* The formatted PDF output of `PDFExtractor` is split into contiguous 8k-character segments, which correspond roughly to a few pages of text and tables. We use zero overlap between segments ($\text{CHUNK_OVERLAP} = 0$) to avoid duplicating table rows across chunks, but this also means that large review tables can be cut exactly at a segment boundary. Although the retrieval step always expands the top retrieved segment to include its two immediate neighbors on each side, tables that span more than this local window, or that reappear as “continued” tables much later in the paper, may still be only partially visible in the model’s context.
- *Embedding sensitivity.* Dense retrieval is driven by an embedding model (`nomic-embed-text`) that maps each segment and each query into a high-dimensional numerical vector so that semantically similar texts lie close to each other. Retrieval quality therefore depends on how well this general-purpose encoder captures specialised ALD/ALE terminology and fine-grained distinctions between process variants, precursor chemistries, or temperature regimes. If the embedding space does not separate these concepts cleanly, segments that contain the relevant table rows may receive lower similarity scores than less relevant segments.
- *Top-k retrieval and local windowing.* For each query we retrieve only the single most similar segment ($\text{TOP_K} = 1$) and then expand around this “anchor” by adding its immediate neighbours. This favours a compact context window and keeps the RAG setting comparable to the full-context Setting 3.1, but it also assumes that most of the evidence the model needs is concentrated in one contiguous region of the paper. Queries that implicitly refer to several distant regions (for example, all SiO_2 processes in a table that is split across pages, or a table plus an explanatory paragraph elsewhere in the text) may require information scattered across non-adjacent segments that this single-anchor strategy never retrieves together.
- *Loss of global document context.* In contrast to Setting 3.1, where the model sees the entire extracted paper at once, the RAG variant deliberately restricts the

model to a narrow slice of the document and instructs it to answer *only* from the retrieved segments. For table-oriented queries that require aggregating over all rows, comparing multiple materials, or cross-checking constraints stated in different parts of the article, this narrow window can omit complementary rows or conditions that happen to fall outside the retrieved neighbourhood. In such cases, the retrieval stage may remove crucial context that the full-context setting would have made available.

This setting thus tests a retrieval-augmented approach designed to reduce context length and emphasize relevant evidence, while also introducing retrieval- and segmentation-dependent behavior that may influence downstream performance.

B Evaluation Metrics for the Neural Q&A Tabular Results

To evaluate RQ2 and RQ3, we measure how closely the tables produced by the neural systems match the SPARQL gold-standard tables on a per-query basis. Intuitively, we want a metric that (i) is insensitive to permutations of rows and columns, (ii) distinguishes between minor numeric or textual deviations and completely wrong entries, and (iii) exposes both *precision* (spurious rows or cells) and *recall* (missing rows or cells). We adopt the *Relative Mapping Similarity* (RMS) metric introduced by Liu et al. [38] as part of Google research, which is specifically designed for table-level evaluation of chart and table question answering systems. Our implementation of the RMS-based evaluation, built on top of the official DePlot metrics, is available at https://github.com/sciknoworg/alld-ale-orkg-review/blob/main/eval_deplot_rms/README.md. This implementation documents how we adapt Google’s DePlot code (<https://github.com/google-research/google-research/tree/master/deplot>) to our ALD/ALE SPARQL and neural querying results dataset provided as the Zenodo dump [15].

Following Liu et al., RMS builds on the earlier Relative Number Set Similarity (RNSS) metric [39, 40], which only compares the unordered set of numeric values in the predicted table against those in the gold table. Let $P = \{p_i\}_{i=1}^N$ and $T = \{t_j\}_{j=1}^M$ denote the predicted and target number sets, respectively. The relative numeric error between two numbers is

$$D(p, t) = \min\left(1, \frac{|p - t|}{|t|}\right). \quad (1)$$

Given the $N \times M$ matrix of pairwise distances $D(p_i, t_j)$, a minimum-cost bipartite matching $X \in \{0, 1\}^{N \times M}$ is found. RNSS is then defined as

$$\text{RNSS} = 1 - \frac{\sum_{i=1}^N \sum_{j=1}^M X_{ij} D(p_i, t_j)}{\max(N, M)}. \quad (2)$$

While RNSS is useful as a numeric sanity check, it ignores the *position* of values in the table (which row/column they belong to), discards all non-numeric content, and does not distinguish precision from recall.

RMS instead views a table as an unordered collection of *mappings* from row and column headers to a cell value. Each entry in the predicted table is represented as $p_i = (p_i^r, p_i^c, p_i^v)$, where p_i^r and p_i^c are the row and column headers, and p_i^v is the cell value; the gold table is represented analogously as $t_j = (t_j^r, t_j^c, t_j^v)$. To compare two entries p_i and t_j , RMS combines (i) a textual similarity between the concatenated keys $p_i^r \| p_i^c$ and $t_j^r \| t_j^c$, and (ii) a similarity between the cell values p_i^v and t_j^v .

Textual similarity is measured with a normalized Levenshtein distance $NL_\tau(\cdot, \cdot)$ with threshold τ , which clips large edit distances to 1:

$$s_{\text{key}}(i, j) = 1 - NL_\tau(p_i^r \| p_i^c, t_j^r \| t_j^c), \quad (3)$$

so that $s_{\text{key}}(i, j) \in [0, 1]$. Numeric values are compared using the relative error with a second threshold θ ; the corresponding similarity is

$$s_{\text{val}}(i, j) = 1 - \min\left(1, \frac{|p_i^v - t_j^v|}{|t_j^v|}\right), \quad (4)$$